

Aufgabe 4: Routing

Es geht um Distance-Vector-Verfahren. Dabei kennt jeder Router nur seine direkten Nachbarn und deren Kosten. Die Router tauschen regelmäßig ihre Routingtabellen (Distanzen) mit den Nachbarn aus und aktualisieren ihre eigene Tabelle, wenn sie über einen besseren Weg zu einem Ziel erfahren.

- Initialisierung:

Jeder Router kennt nur die direkten Nachbarn und die Kosten zu ihnen. Alle anderen Wege sind unendlich oder unbekannt.

- Regelmäßiger Austausch:

Jeder Router schickt seine Routingtabelle an alle direkten Nachbarn (Distance Vectors).

- Aktualisierung:

Für jeden empfangenen Eintrag vom Nachbarn:

- Berechne:
neue Distanz = Kosten zum Nachbarn + dessen Kosten zum Ziel
- Falls neue Distanz < bekannte Distanz:
→ Update: trag neue Distanz und neuen via-Weg ein.

- **Wiederhole den Austausch**, bis keine Änderungen mehr auftreten, d.h. stabiler Zustand erreicht.

Unterschiede:

Distanz-basiert

- Distanz-Vektor
- Bellman-Ford-Algorithmus
- Router speichern Distanzen zu bekannten Netzen
- Periodischer Abgleich der Tabellen
- Eingeschränkte Sicht auf Netz

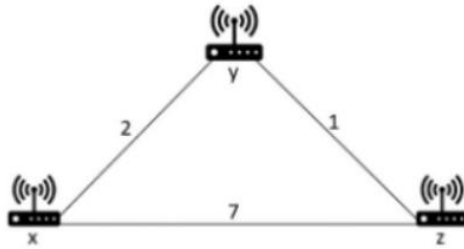
Link-basiert

- Link-State
- Router speichern aktuelle Karte des Netzes
- Wahl des kürzesten Pfads zum Ziel
- Periodischer Abgleich der Karten
- Vollständige Sicht auf Netz (wenn auch veraltet)

Das Distance-Vector-Verfahren ist einfacher und benötigt weniger Speicher, funktioniert aber nur durch ständiges Austauschen und Vergleichen mit Nachbarn.

Im Gegensatz dazu bauen Link-State-Router eine vollständige Karte des Netzes auf und berechnen zentralisiert die kürzesten Pfade.

a) Bestimmen Sie die Routingtabellen in jedem Schritt, indem Sie Ihren zuvor entwickelten Algorithmus anwenden. Geben Sie danach den kostengünstigsten Weg von Router „z“ zu Router „x“ an. Sie können die Lösungen einfach direkt in die Tabellen der PDF einfügen und diese in den Pull Request hinzufügen.



Von x	Via x	Via y	Via z
Zu x			
Zu y		2	
Zu z			7

Von y	Via x	Via y	Via z
Zu x	2		
Zu y			
Zu z			1

Von z	Via x	Via y	Via z
Zu x	7		
Zu y		1	
Zu z			

Von x	Via x	Via y	Via z
Zu x			
Zu y		2	8
Zu z		3	7

Von y	Via x	Via y	Via z
Zu x	2		8
Zu y			
Zu z	9		1

Von z	Via x	Via y	Via z
Zu x	7	3	
Zu y	9	1	
Zu z			

Von x	Via x	Via y	Via z
Zu x			
Zu y		2	8
Zu z		3	7

Von y	Via x	Via y	Via z
Zu x	2		4
Zu y			
Zu z	5		1

Von z	Via x	Via y	Via z
Zu x	7	3	
Zu y	9	1	
Zu z			

Weg von z zu x via y kostet 3

b) Die Kosten zwischen „x“ und „y“ steigen nun von 2 auf 7. Berechnen Sie die Routingtabellen mit Hilfe des Algorithmus. Ändert sich der kostengünstigste Pfad von „z“ nach „x“?

ja, der kostengünstigste Weg von z nach x ist direkter Weg mit Kosten 7.

Von x	Via x	Via y	Via z
Zu x			
Zu y		7	
Zu z			7

Von y	Via x	Via y	Via z
Zu x	7		
Zu y			
Zu z			1

Von z	Via x	Via y	Via z
Zu x	7		
Zu y		1	
Zu z			

Von x	Via x	Via y	Via z
Zu x			
Zu y		7	8
Zu z		8	7

Von y	Via x	Via y	Via z
Zu x	7		8
Zu y			
Zu z	14		1

Von z	Via x	Via y	Via z
Zu x	7	8	
Zu y	14	1	
Zu z			

Von x	Via x	Via y	Via z
Zu x			
Zu y		7	8
Zu z		8	7

Von y	Via x	Via y	Via z
Zu x	7		8
Zu y			
Zu z	14		1

Von z	Via x	Via y	Via z
Zu x	7	8	
Zu y	14	1	
Zu z			

c) Wenn Router D ausfällt, kann nur Router C das zunächst bemerken, da er direkt mit D verbunden ist. Router C erkennt den Ausfall beim nächsten Kommunikationsversuch, z. B. durch Timeout. Es passiert aber nicht automatisch. Die Distanzmatrix bleibt unverändert, solange keine Aktualisierung erfolgt. Erst wenn der Algorithmus erneut ausgeführt wird, wo schon D nicht mehr existiert, erkennen auch Router A und B, dass keine Verbindung mehr zu D besteht.